

Sesión 5

Aprendizaje Federado



Entrenar un modelo compartido sin centralizar los datos: el modelo viaja, los datos se quedan.

Agenda de hoy

0 : 00		Recap y encuadre Dónde estamos en la plataforma
0 : 10		El dilema del dato Por qué a veces no se puede (ni conviene) centralizar
0 : 30		¿Qué es el aprendizaje federado? Mover el cómputo al dato · arquitectura
0 : 55		FedAvg, paso a paso Rondas de comunicación y agregación
1 : 25		Desafíos Datos non-IID · costo de comunicación · privacidad real
1 : 50		Federado en MLOps Cross-device vs cross-silo · frameworks
2 : 10		Práctica Simular clientes y servidor · IID vs non-IID
2 : 45		Mini-TP 5 y cierre Trade-off privacidad vs performance

La plataforma ya sirve y reacciona; hoy aprende sin centralizar el dato

REST

GraphQL

gRPC

Streaming

Federado

Cloud/Lake

Seguridad

Integra

En las Sesiones 1–4

- REST, GraphQL y gRPC: cómo se habla la plataforma
- Streaming: datos en movimiento en tiempo real
- Siempre asumimos los datos juntos, en un lugar

Hoy resolvemos

- Los datos están repartidos entre muchos 'clientes'
- No se pueden (ni conviene) mover a un servidor central
- Aun así queremos un modelo que aprenda de todos → federado

¿Y si el dato no puede salir de donde está?

Miles de millones de teléfonos, sensores y organizaciones generan datos valiosos. Pero centralizarlos no siempre es posible ni deseable.



Privacidad y regulación

Datos personales o sensibles (salud, finanzas) que no deben salir del dispositivo u organización.



Latencia y conectividad

Procesar en el dispositivo mejora la latencia y funciona offline.



Costo y batería

Subir todo a la nube es caro en ancho de banda y energía.



El dato es más relevante local

Lo que el usuario escribe o mide en su equipo describe mejor su caso que un dataset central.

La idea: mover el cómputo al dato

En lugar de traer los datos al modelo, llevamos el modelo a los datos. Cada cliente entrena con lo suyo y solo comparte lo aprendido (los parámetros), nunca los datos brutos.

Centralizado (lo habitual)

Se copian TODOS los datos a un servidor y ahí se entrena. Simple, pero exige mover y concentrar datos sensibles.

`dato → servidor → modelo`

- Todo el dato en un solo lugar
- Riesgo de privacidad y cumplimiento
- Un punto único de fuga

Federado

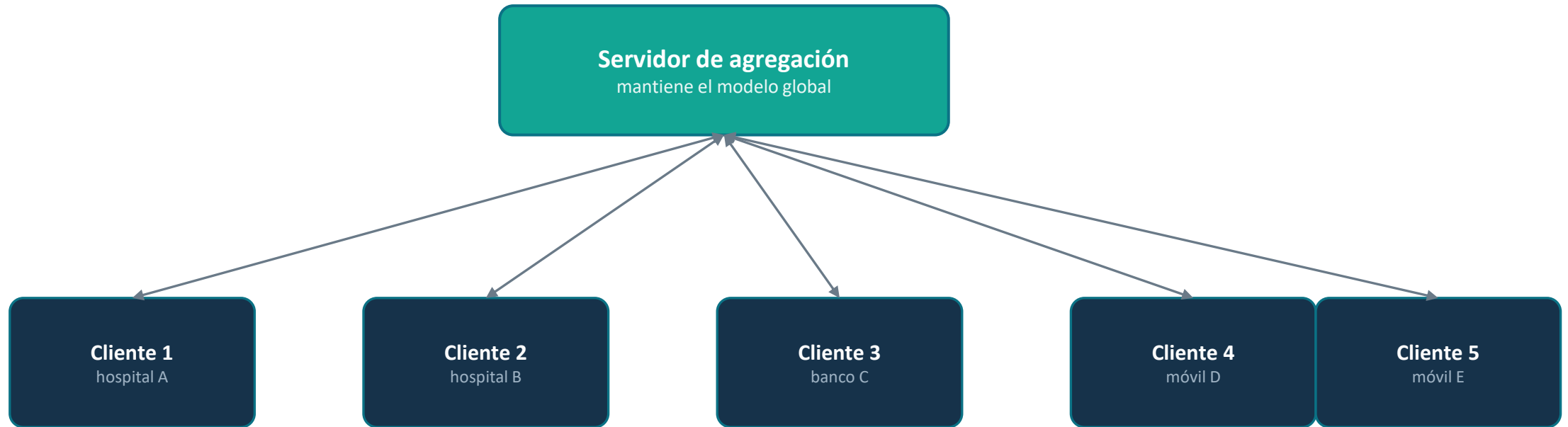
El modelo viaja a cada cliente, entrena localmente y devuelve solo sus parámetros. El servidor los combina en un modelo global.

`modelo → dato (local) → parámetros → global`

- El dato nunca sale del cliente
- Mejor privacidad por diseño
- Aprende de todos sin verlos

Arquitectura: clientes, servidor y rondas

Un servidor de agregación coordina; muchos clientes entrenan en paralelo con sus propios datos. Se avanza en rondas de comunicación.



↓ el servidor envía el modelo · ↑ los clientes devuelven parámetros (no datos)

Cross-device: muchísimos móviles, poco datos c/u. Cross-silo: pocas organizaciones (salud, finanzas), mucho dato c/u.

FedAvg: promedio federado, paso a paso

- 1 El servidor envía el modelo global a un subconjunto de clientes
- 2 Cada cliente entrena unas épocas con SUS datos (SGD local)
- 3 Cada cliente devuelve solo sus parámetros actualizados
- 4 El servidor los promedia, ponderando por cantidad de datos
- 5 Se repite por muchas rondas hasta converger

La agregación

$$\mathbf{w}_{\text{global}} = \sum_k (n_k/n) \cdot \mathbf{w}_k$$

El modelo global es el promedio de los modelos locales, pesado por cuántos datos tiene cada cliente. Nunca se ven los datos: solo los pesos.

Dónde se usa de verdad



Teclado del móvil

Predecir la próxima palabra aprendiendo de lo que escribes, sin subir tus mensajes.



Salud

Varios hospitales entrenan un modelo común sin compartir historias clínicas.



Finanzas

Bancos detectan fraude en conjunto sin exponer las transacciones de sus clientes.



Recomendación

Personalizar en el dispositivo sin centralizar el comportamiento de cada usuario.

Una buena aplicación: el dato local es más relevante que el central, la privacidad importa, y las etiquetas se infieren del uso.

El principal: datos non-IID (heterogéneos)

En federado, cada cliente ve una distribución distinta de datos (non-IID). Eso hace que los modelos locales 'tiren' para lados diferentes y el promedio converja peor.

IID (ideal)

Cada cliente tiene una muestra parecida del total (todas las clases, balanceadas).

- Los modelos locales apuntan al mismo lado
- El promedio funciona casi como centralizado
- Convergencia estable

Non-IID (realista)

Cada cliente ve datos sesgados (un hospital, un idioma, una región): distribuciones muy distintas.

- Los modelos locales divergen entre sí
- El promedio pierde accuracy y oscila
- Se mitiga con más rondas, menos pasos locales, FedProx...

Los otros tres a tener en cuenta



Costo de comunicación

Cada ronda mueve parámetros por la red. Se reduce con menos rondas, compresión o cuantización.



Clientes intermitentes

Móviles que entran y salen, con distinto poder de cómputo (stragglers y heterogeneidad de sistema).



La privacidad no es gratis

Los propios parámetros/gradientes pueden filtrar información sobre los datos.

Por eso 'federado' no es automáticamente 'privado': hace falta reforzarlo.

¿Es privado de verdad? Cómo reforzarlo



Privacidad diferencial

Se agrega ruido calibrado a los parámetros antes de compartirlos. Garantiza que la contribución de un individuo no se pueda distinguir, a costa de algo de precisión.

Introducción hoy; se profundiza en la Sesión 7 (seguridad).



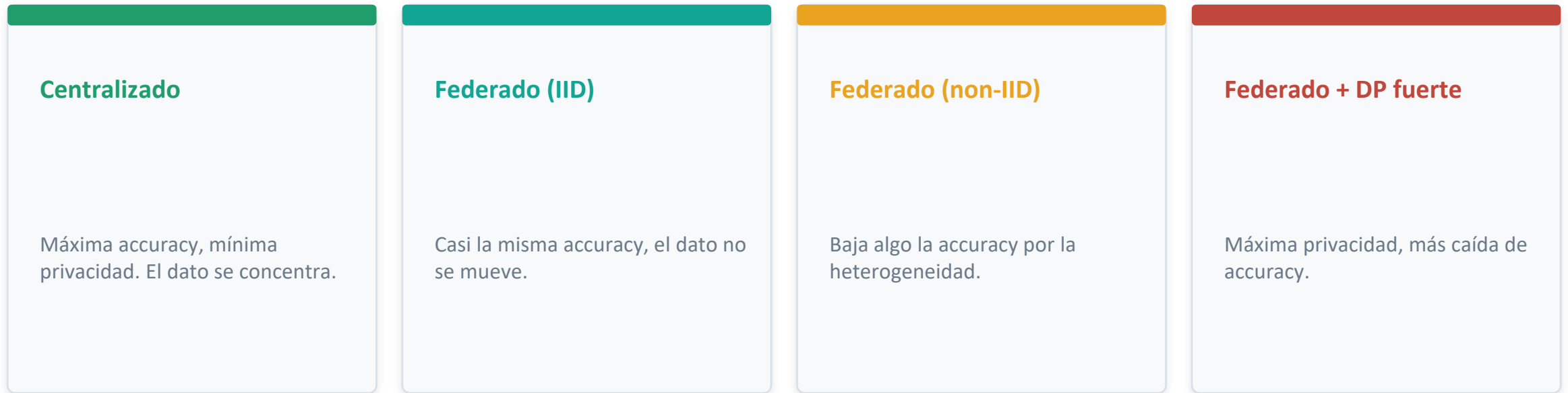
Agregación segura

El servidor solo puede ver la SUMA de los parámetros de todos, nunca el de un cliente individual. Se logra con criptografía (secure aggregation).

Federado + DP + agregación segura = privacidad por capas.

El trade-off: privacidad vs performance

Cada medida que suma privacidad (federar, agregar ruido, comunicar menos) suele restar algo de accuracy. El trabajo de MLOps es encontrar el punto útil.



El Mini-TP de hoy mide exactamente este trade-off sobre tu propio modelo.

Frameworks de aprendizaje federado

Framework	Enfoque	Cuándo
Flower (flwr)	Agnóstico del modelo (PyTorch, TF, sklearn); simulación y despliegue real	El más práctico para empezar y para la práctica de hoy
TensorFlow Federated	Integrado al ecosistema TensorFlow/Keras	Si ya trabajas todo en TF
PySyft (OpenMined)	Foco en privacidad: DP y computación segura	Cuando la privacidad formal es el centro

Hoy vemos el algoritmo desde cero (para entenderlo) y luego cómo se ve con Flower. La idea de FedAvg es la misma en todos.

Cómo encaja en la plataforma



Entrenar sobre los 'clientes'

Cada sucursal, dispositivo u organización aporta sin ceder sus datos brutos.



El servidor de agregación

Vive en tu plataforma; produce el modelo global ronda a ronda.



Se integra con lo que ya tienes

El modelo global se versiona en MLflow y se guarda en MinIO, como cualquier otro.



Se sirve igual

Una vez entrenado, se expone por REST/gRPC como en las sesiones anteriores.

Es una capa de entrenamiento distinta, pero el resultado (un modelo) se opera con el mismo pipeline del bimestre.

Qué construimos

Una simulación federada desde cero: repartimos un dataset entre varios 'clientes', entrenamos localmente, agregamos con FedAvg y comparamos contra el entrenamiento centralizado. Todo corre en el notebook.



NOTEBOOK `federated_tutorial.ipynb`

Repartir en clientes y entrenar local

```
# repartir el dataset entre K clientes (IID)
clientes = particionar(X_train, y_train, K=5)

# entrenamiento local: unas épocas de SGD
def entrenar_local(w_global, Xc, yc, epocas=3):
    w = copiar(w_global)
    for _ in range(epocas):
        grad = gradiente(w, Xc, yc)
        w = w - lr * grad
    return w, len(Xc)  # pesos + n° de datos
```

Qué mirar

- Cada cliente parte del MISMO modelo global
- Entrena solo con sus datos
- Devuelve pesos, nunca los datos
- Guardar len(Xc) para ponderar

NOTEBOOK federated_tutorial.ipynb

La agregación FedAvg en el servidor

```
def fedavg(actualizaciones):  
    # actualizaciones = [(w_k, n_k), ...]  
    n = sum(n_k for _, n_k in actualizaciones)  
    w_global = sum((n_k/n) * w_k  
                   for w_k, n_k in actualizaciones)  
    return w_global  
  
# una ronda de comunicación  
ups = [entrenar_local(w_global, Xc, yc)  
        for Xc, yc in clientes]  
w_global = fedavg(ups)    # repetir R rondas
```

Qué mirar

- El servidor solo promedia pesos
- Ponderado por datos de cada cliente
- Una ronda = local + agregar
- Con más rondas, se acerca al centralizado

NOTEBOOK federated_tutorial.ipynb

IID vs non-IID y comparación

```
# partición sesgada: cada cliente, pocas clases
clientes_niid = particionar_no_iid(X, y, K=5)

acc_central = entrenar_centralizado(X, y)
acc_iid      = federado(clientes,      R=20)
acc_niid     = federado(clientes_niid, R=20)
# graficar accuracy vs ronda para cada caso
```

La conclusión

Federado con datos IID alcanza casi la accuracy del centralizado sin mover un solo dato. Con datos non-IID converge más lento y algo peor: ese es el precio de la heterogeneidad.

NOTEBOOK federated_tutorial.ipynb

Errores comunes (y cómo evitarlos)



Compartir datos 'sin querer'

Solo se envían parámetros; nunca muestras ni features crudas.



Ignorar el non-IID

Probar siempre con datos sesgados: es lo realista.



Demasiados pasos locales

Con non-IID, muchos pasos locales hacen divergir; baja las épocas.



No ponderar por datos

FedAvg pesa por nº de datos de cada cliente; olvidarlo sesga.



Crear que federado = privado

Sin DP/agregación segura, los pesos pueden filtrar.



Simulación que no corre

Fija versiones y semilla; el notebook debe correr de punta a punta.



MINI-TP 5 · INDIVIDUAL · ENTREGA ESTA SEMANA

Federa tu modelo y mide el trade-off

Consigna

- Reparte tu dataset (o el provisto) entre K clientes y corre una simulación federada con FedAvg.
- Compara la accuracy contra el entrenamiento centralizado.
- Repite con una partición non-IID y observa la diferencia.
- Analiza el trade-off privacidad vs performance: ¿cuánto cuesta no mover el dato?
- (Opcional) Agrega ruido (DP simple) y mide el impacto.

Se evalúa

- Corre de punta a punta
- FedAvg bien implementado
- Compara IID / non-IID / central
- Reflexión sobre el trade-off

Actividad: `mini_tp5_federado_actividad.ipynb`
(starter con celdas TODO).

Dónde quedó la plataforma

REST

GraphQL

gRPC

Streaming

Federado

Cloud/Lake

Seguridad

Integra

Hoy logramos

- Entrenar un modelo sin centralizar los datos
- Implementar FedAvg y ver IID vs non-IID
- Medir el trade-off privacidad vs performance



Próxima · Sesión 6

Nube y Data Lakes: dónde viven datos, modelos y artefactos (MinIO/S3), integrados con Airflow y MLflow. Y llega el Hito TP #2: checkpoint del integrador, evaluado en clase.

Para la próxima: Mini-TP 5 corriendo y el integrador listo para el checkpoint.

Referencias y recursos

Herramientas y código

- Flower — flower.ai/docs · paquetes: `uv add flwr scikit-learn numpy matplotlib`
- TensorFlow Federated — tensorflow.org/federated · PySyft (OpenMined) — openmined.org
- Repositorio del curso: `clase5/Practica (federated_tutorial.ipynb, mini_tp5_federado_actividad.ipynb)`.

Lectura

- McMahan et al. (2017) — Communication-Efficient Learning of Deep Networks from Decentralized Data (el paper de FedAvg).
- Kairouz et al. (2021) — Advances and Open Problems in Federated Learning (panorama).